

Algorithms for Data Science
Exam 2025
Group **H**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. 5 points Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1. Coordinates may be arbitrarily large and all start at height 0. The function `blocks(pos, h)` takes the list `pos` of drop positions and the target height `h`, and returns the x-coordinate where a column's height first reaches `h`.

Hint: the task is related to the third programming task.

```

1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7          # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10             # ??? [Line 10]
11             return x

```

Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

- | | |
|---------------------------------------|--|
| • Line 3: <u> E </u> | • Line 9: <u> D </u> |
| • Line 6: <u> F </u> | |
| • Line 7: <u> N </u> | • Line 10: <u> G </u> |

Code snippets to choose from:

- | | |
|---|--|
| • (A) <code>heights = [0] * max(pos)</code> | • (I) <code>if x < len(heights):</code> |
| • (B) <code>heights[i] += x</code> | • (J) <code>if heights[x] < h:</code> |
| • (C) <code>if heights[x] == 0:</code> | • (K) <code>if heights[h] == i:</code> |
| • (D) <code>heights[x] += 1</code> | • (L) <code>heights[x] += i</code> |
| • (E) <code>heights = {}</code> | • (M) <code>heights[i] = x</code> |
| • (F) <code>if x not in heights:</code> | • (N) <code>heights[x] = 1</code> |
| • (G) <code>if heights[x] == h:</code> | • (O) <code>heights = [0] * n</code> |
| • (H) <code>heights[x] = h</code> | |

Note: The indentation of your code is the same as of `# ???`. In other words, your code for an empty line will start at the position of `#`.

2. 4 points Complete the following algorithm **AlgoA**($a[1..n]$) such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 14T(\lceil n/26 \rceil) + \Theta(n^{12}) .$$

You don't need to understand what the algorithm computes!

The input of **AlgoA**($a[1..n]$) is an array a of n integers.

The input of the auxiliary function **Copy_Sub_Array**($a[1..n], i, m$) is an array $a[1..n]$, a position i and a length m . The function returns a copy of the subarray $a[i+1..i+m]$.

<pre> AlgoA($a[1..n]$) ┌ if $n < 27$ then │ return 13 └ else $d = \underline{\hspace{1cm}13\hspace{1cm}}$ $m = \lceil n / (2 \cdot d) \rceil$ $sum = 0$ for $i = 1$ to 6 do $b = \text{Copy_Sub_Array}(a, i, m)$ $sum = sum + \text{AlgoA}(b)$ for $i = 1$ to $\underline{\hspace{1cm}8\hspace{1cm}}$ do $b = \text{Copy_Sub_Array}(a, i, m)$ $sum = sum + \text{AlgoA}(b)$ $w = 1$ for $j = 1$ to $\underline{\hspace{1cm}12\hspace{1cm}}$ do $w = w \cdot n$ // Time: $O(1)$ for $k = 1$ to w do $sum = sum + a[27]$ return sum </pre>	<pre> Copy_Sub_Array($a[1..n], i, m$) ┌ $b[1..m] =$ new empty array │ for $j = 1$ to m do │ $b[j] = a[i + j]$ └ return b </pre>
---	---

Fill in the three blanks in the algorithm above, each time writing a **single appropriate integer number!**

3. 4 points Solve the following recurrence using the master theorem. Determine (for yourself) the values for a , b , and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 64 \cdot T(n/2) + \Theta(n^8)$$

Your answers:

$$c = \underline{\mathbf{6}}$$

Case: **C**

Resulting running time $T(n) = \underline{\hspace{2cm} \Theta(n^8) \hspace{2cm}}$

4. Simplify the following running times as far as possible:

(a) 2 points $\Theta(4 \cdot n^{13/16} + 4 \cdot n^{6/8} \cdot n^{2/8}) = \Theta(\underline{\hspace{2cm} n \hspace{2cm}})$

(b) 2 points $\Theta((15^{\log_{15} n})^{15} + (5^{n/15})^5) = \Theta(\underline{\hspace{2cm} 5^{n/3} \hspace{2cm}})$

(c) 2 points $\Theta(7 \log_2 n + 5 \cdot n^5 + n^4 \log_2 n) = \Theta(\underline{\hspace{2cm} n^5 \hspace{2cm}})$

5. 5 points Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 100, 70, 50, 95, 65, 60, 40, 30, 90, 75, 55, 65, 40, 15, 45, 35, 20, 25, 80, 85 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?

(Write a single number between 1 and 21; there is only one correct answer.)

Your answer: $j =$ 8 (the violating edge's other endpoint is at 16.)

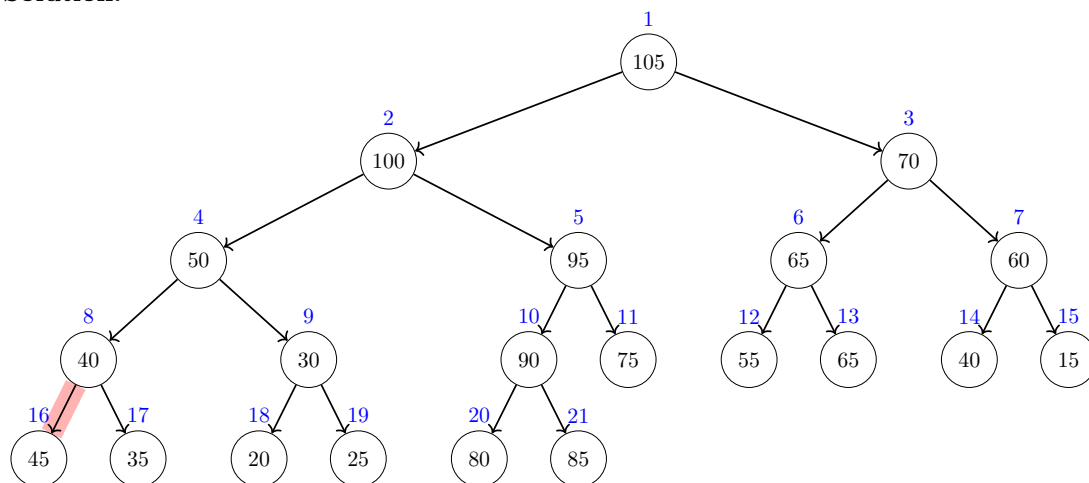
What is the new value for $a[j]$?

(Write a single number, as large as possible and larger than the old value of $a[j]$.)

Your answer: $a[j] =$ 50 (the largest value from $[45, 50]$)

Your tree should contain **all** the elements of the array, from $a[1] = 105$ up to $a[21] = 85$:

Solution:



*Explanation (not required): The heap invariant is violated at the red edge as the parent (index **8**) has a smaller value than its child (index **16**). To fix the heap invariant, it suffices to increase $a[8]$ to any value between 45 and 50. The largest possible value (as asked in the task) is thus 50.*

6. 9 points Dijkstra's algorithm (see below) is run on the following graph where the start node is s .

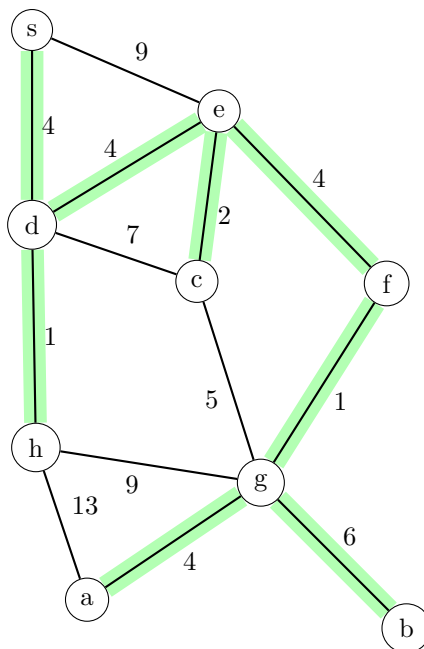
What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```

dist[0..n-1] filled with INF
parent[0..n-1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v,w) do
            if dist[w] > dist[v] + cv,w then
                dist[w] = dist[v] + cv,w
                parent[w] = v
                q.add(w, -dist[w])

```



Solution:

	a	b	c	d	e	f	g	h	s
distance	17	19	10	4	8	12	13	5	0
parent	g	g	e	s	d	e	f	d	NONE

Note:

- The entries of the parent table consist of the names of the nodes or the special value *NONE*. For example, the parent of some node could be s whereas the parent of s is *NONE*.
- If a vertex has more neighbors, we consider them **in lexicographic order** (that is, node a would come before node b and so on)!

8. 4 points Add the following keys (in the order as given) into the hash table below using the hash function

$$h(k) = k \bmod 14$$

Keys: 5, 8, 19, 33

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:

Hash table:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
14					47			36					
0					5			22					
					19			8					
					33								
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

9. 4 points What is the most appropriate data structure for the following scenario, chosen from the list below?

Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

You are building the warehouse's inventory log. Each new transaction has a unique integer ID and must be added when it occurs. You never need to look up a specific ID, but you sometimes need to list all transaction IDs in the order they happened.

1. Array indexed by keys
2. AVL tree
3. Find-union
4. Hash table
5. Heap/Priority queue
6. Sorted Linked list
7. Queue
8. Stack
9. Sorted array
10. Unsorted array

Most appropriate data structure: **Unsorted array, with modifications: stack, queue**

Explanation:

Solution: We use a simple array: each new ID is appended at the end, and reporting all IDs just reads the array from start to finish.

The memory usage is optimal at $\Theta(n)$, where n is the number of elements. Given the guarantee that **Add** is called only on new keys, insertions take amortized constant time by appending at the end, which makes the total time of **Add** to be $\Theta(n)$. Furthermore, we can output all the elements in their insertion order (in total $\Theta(n)$ time) by going through the unordered array from the beginning to the end.

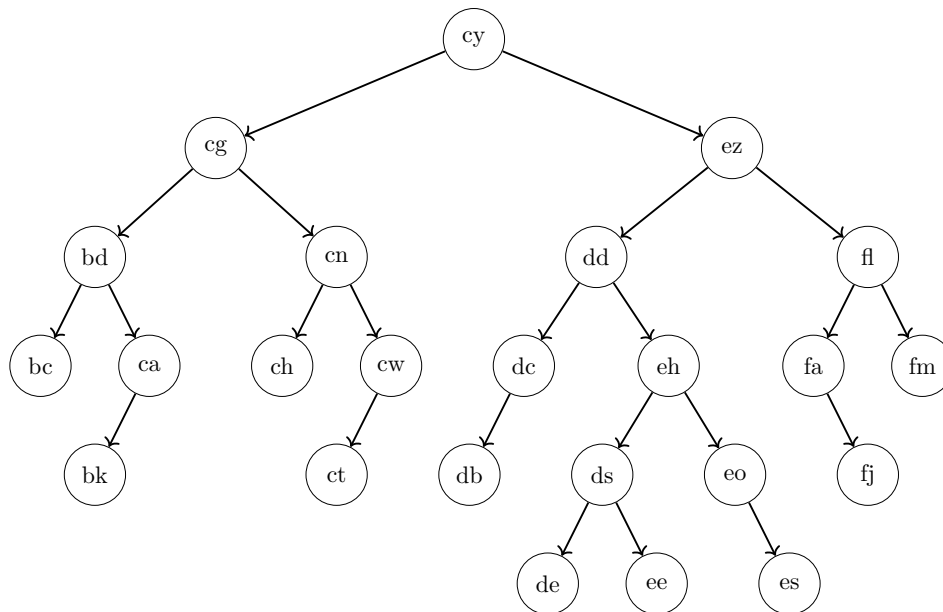
All the other data structures are worse under this scenario as they cannot return all the elements in their insertion order. The only exception are stack and queue, but they don't provide such a functionality out of the box, but it could be achieved by using another auxiliary queue (or two auxiliary stacks) to scan through all the elements without losing those elements.

10. 5 points What is the height of the node *cg* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes on a longest path starting at v within the subtree of v (not counting the dummy “NONE” nodes that have height 0).

Height of *cg*: 4 (Write only a single number.)

Height of the root: 6 (Write only a single number.)



Is the AVL invariant violated? Your answer: no (Write “yes” or “no”.)

Write the names of two nodes,

$X =$ bk and $Y =$ bd, such that the following holds:

- If the tree violates the AVL invariant, then the violation occurs at node X (caused by the heights of X 's children), and adding a new child to node Y will restore the AVL invariant.
- Otherwise, if the tree satisfies the AVL invariant, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y 's children).
- If there is more than one valid choice for X , choose the one with the alphabetically **smallest** name. **After selecting X** , if there is more than one valid choice for Y , choose the one with the alphabetically **smallest** name. (Example: “bc” is alphabetically smaller than “fm”.)

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X . If there are multiple valid choices for X , choose the one with the alphabetically smallest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

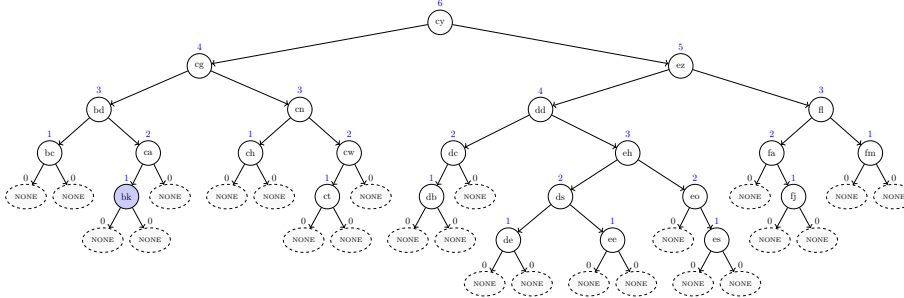
Explanation:

The tree satisfies the AVL invariant. There are seven different candidates for X , where adding a node leads to a violation of the AVL invariant:

bk, ct, db, de, ee, es and fj.

Recall that out of these different possibilities, the task asks you to choose the one where X is the alphabetically smallest node; hence, $X = \text{bk}$.

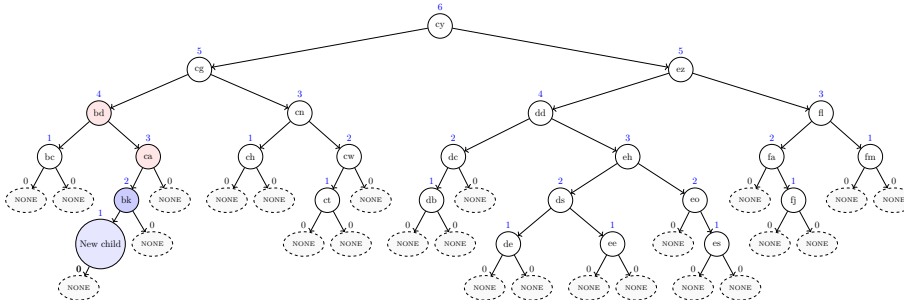
The first tree shows the heights and dummy nodes, as well as the node $X = \text{bk}$ (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the two violating parent nodes in red (whose children differ in height by more than 1):

ca and bd.

Among these violating nodes, the task asks you to choose the alphabetically smallest one; hence $Y = \text{bd}$.



For completeness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

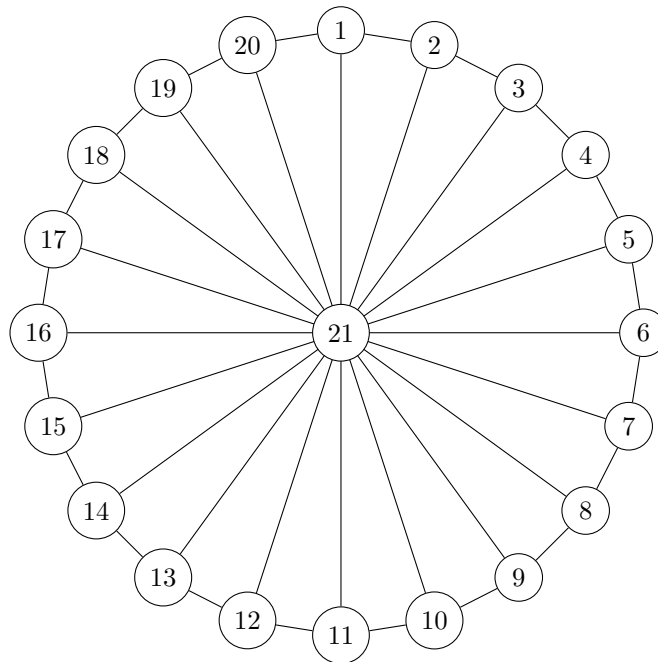
- Candidate ct → cw and cn
- Candidate db → dc
- Candidate de → dd, ez and cy
- Candidate ee → dd, ez and cy
- Candidate es → eo, dd, ez and cy
- Candidate fj → fa and fl

11. 5 points Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to $n=21$ and each edge has the same weight 1. Observe that the vertices 1 to 20 lie on a big cycle and each of them is connected to the center vertex 21 via an edge.

Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 16 iterations.

```

1  $\delta[1..21][1..21]$  = new 2D array filled with  $+\infty$ 
2 for  $i = 1$  to 21 do
3    $\delta[i][i] = 0$ 
4 foreach  $edge(v, w, c)$  do
5    $\delta[v][w] = c$ 
6    $\delta[w][v] = c$ 
7 for  $k = 1$  to 16 do
8   foreach pair  $(v, w)$  do
9      $\delta[v][w] = \min(\delta[v][w], \delta[v][k] + \delta[k][w])$ 
10 return  $\delta$ 
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 16 times**!

Recall that the weight of every edge is equal to 1.

If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.

$$\delta[20][17] = \underline{\quad 17 \quad}$$

$$\delta[5][14] = \underline{\quad 9 \quad}$$

$$\delta[19][21] = \underline{\quad 1 \quad}$$

$$\delta[19][16] = \underline{\quad \text{INF} \quad}$$

$$\delta[6][18] = \underline{\quad \text{INF} \quad}$$

12. 4 points Consider the following algorithm.

```
Input: integer  $l$   
 $z = 2$   
 $e = 0$   
while  $l > l$  do  
     $e = e + 6$   
     $g = -4$   
     $z = z + g$   
     $e = e + g$   
     $z = z + 18$ 
```

Someone wrote the following invariant for the loop:

$$Y \cdot z = U \cdot e + J$$

with the three constants

- $Y = 8$
- $U = 42$
- $J = 12$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, Y , U , or J has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? **Y** (*Write the name of one of the three constants.*)

The new value for the chosen constant: **6**

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

Your network administrator has n Boolean firewall rules, each can be enabled or disabled. The overall security policy is given as a complex logical formula over these rule variables using “and”, “or”, and “not”. Is there an assignment of enable/disable values that makes the policy true?

Consider the following decision problem B:

You’re organizing a tour of n meeting rooms connected by corridors for a conference. You need to start at the lobby, visit the keynote hall, and return to the lobby, with no room visited more than twice. Is such a tour possible?

Mark all the correct sentences and give a short explanation.

- ☐ A can be reduced to B in polynomial time.
- ☒ **B can be reduced to A in polynomial time.**
- ☐ A is in P.
- ☒ **B is in P.**
- ☒ **It is unknown whether A can be reduced to B in polynomial time.**
- ☐ It is unknown whether B can be reduced to A in polynomial time.
- ☒ **It is unknown whether A is in P.**
- ☐ It is unknown whether B is in P.

Solution: Problem A is equivalent to the NP-complete problem SAT (by modeling each rule as a Boolean variable, the security policy as a logical Boolean SAT formula, and asking whether the formula is satisfiable), whereas problem B can be solved in polynomial time: we interpret rooms as vertices and corridors as edges; the question reduces to whether there exists a simple path connecting the lobby and the keynote hall (walking this path forth and back gives us a tour visiting each room at most twice); reachability can be checked in polynomial time via breadth-first search (BFS).

Consequently, as problem B can be solved in polynomial time, it can be reduced in polynomial time to any other problem, thus also to A.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether A is in P, and, consequently, whether there exist a polynomial time reduction from A to any polynomial-time problem (e.g., to B).